

TFG - XplicaVoz

Jorge Aured Zarzoso, Germán Bravo Quintián,
Rafael López Rodríguez, Iván Pastor Sacristán

12 de marzo de 2026

Índice

1. Introducción	3
2. Estado del arte	3
2.1. Modelos de clasificación	3
2.1.1. Perceptrón multicapa - MLP	3
2.1.2. Wav2Vec2	4
2.1.3. Random Forest	5
2.1.4. LightGBM	6
2.1.5. Support Vector Machines (SVM)	8
2.1.6. AASIST	10
2.2. Modelos de explicación	12
2.2.1. SHAP	12
2.2.2. LIME	12
2.2.3. Grad-CAM	12
2.2.4. Integrated Gradients	13
2.2.5. DICE	13
2.2.6. SincNet	13
2.2.7. ProtoPNet	13
3. Metodología	13
3.1. Dataset	13
3.2. Extracción de características	14
3.3. Preprocesamiento	18
3.4. Modelos de clasificación	18
3.4.1. Perceptrón multicapa	18
3.4.2. Fine-tuning Wav2Vec2	20
3.4.3. Random Forest	21
3.4.4. LightGBM	25
3.4.5. Support Vector Machine (SVM)	29
3.4.6. AASIST	32
3.5. Modelos de explicación	38

3.5.1. Random Forest	38
--------------------------------	----

1. Introducción

En los últimos años, hemos visto un aumento considerable en el uso de la IA para suplantar la identidad de alguien de forma maliciosa clonando su voz. Estos son los conocidos *deepfakes*. El reconocimiento de dichos *deepfakes* es de vital importancia, pues suponen una gran amenaza. Por otro lado, las técnicas de detección de *deepfakes* no han avanzado al mismo ritmo, y esto lo vemos en la gran cantidad de estafas que se producen de este tipo.

Además, las personas ya no somos capaces de diferenciar entre una voz real y una generada, como se ve en [? ?], donde hacen sendos estudios exponiendo a personas reales a diversos audios, algunos reales, algunos generados y algunos que clonan voces reales. En dichos estudios, se ve que los que son 100 % generados nos cuesta poco detectarlos, pero entre los reales y los clones no diferenciamos bien, y rondamos el 60 % de precisión clasificando dichas voces en reales o falsas. Por esto, no podemos fiarnos más de nuestras capacidades a la hora de diferenciar cuando una voz es real o no.

En este texto, se proponen varias técnicas que podrían ser útiles para la detección de audios generados por IA. Para esta tarea es fundamental sacar las características de la voz, así como gráficas, para poder entrenar los modelos con dichos datos. Entre los audios debe haber tanto reales como generados, y estar medianamente equilibrados, para que los modelos aprendan las diferencias, y así sepan clasificar un audio que nunca han visto.

Pero el rendimiento del modelo de clasificación no es lo único que nos interesa. Hoy en día, los modelos de IA se han convertido en *cajas negras*, donde no sabemos cómo hace las predicciones, dando lugar a problemas de confianza de los humanos respecto de dichas predicciones, como es en el ámbito médico o ingenieril, donde las decisiones que tomen las IAs deben ser precisas, puesto que la vida de las personas puede estar en juego. Para solventar este problema, ha surgido la IA explicable (XAI), la cual toma dichos modelos *caja negra* y da una explicación de qué características de los datos afectan más a la decisión del modelo. En esta investigación, vamos a aplicar XAI para ver qué afecta más a la clasificación de los audios dentro de los distintos grupos que mencionamos antes.

2. Estado del arte

2.1. Modelos de clasificación

2.1.1. Perceptrón multicapa - MLP

Un perceptrón multicapa es una red neuronal que se utiliza en aprendizaje supervisado en el campo del aprendizaje automático. El MLP está compuesto por múltiples capas de neuronas interconectadas, en las que las salidas de las neuronas de una capa se convierten en entradas para la siguiente capa. La primera capa se llama capa de entrada, la última capa se llama capa de salida y las capas intermedias se llaman capas

ocultas. El MLP presenta una gran capacidad para modelar relaciones no lineales y para generalizar. Como punto negativo del uso de este modelo, podemos remarcar la necesidad de un gran conjunto de datos de entrenamiento para evitar el sobreajuste. Por último, cabe resaltar que los MLP's fueron los precursores de otras redes neuronales más complejas como la redes convolucionales y las redes recurrentes. (Fuente: <https://gamco.es/glosario/perceptron-multicapa-mlp/>)

2.1.2. Wav2Vec2

Wav2Vec2 es un modelo de aprendizaje profundo end-to-end para procesamiento de voz que permite obtener representaciones acústicas de alta calidad directamente desde la señal cruda de audio, sin necesidad de características manuales. Fue propuesto por Meta en 2020.

Como parte del entrenamiento, el modelo aprende unidades de habla discreta mediante un gumbel softmax (técnica utilizada en redes neuronales para aproximar el muestreo de distribuciones categóricas discretas de forma diferenciable) para representar las representaciones latentes en la tarea contrastiva.

Tras el preentrenamiento en el habla no etiquetada, el modelo se ajusta con datos etiquetados con una pérdida de Clasificación Temporal Conexiónista (CTC: algoritmo y función de pérdida utilizado en redes neuronales profundas para entrenar modelos de aprendizaje automático en tareas de modelado de secuencias cuando los datos de entrada y salida no están alineados) para ser utilizada en tareas posteriores de reconocimiento de voz.

El modelo se compone de un codificador de características convolucional multicapa que toma la entrada cruda de audio y devuelve representaciones latentes de voz para T instantes de tiempo. Luego son pasados a un transformer para construir representaciones capturando información de toda la secuencia.

El codificador de características consiste en muchos bloques convolucionales seguidos de una capa de normalización y una función de activación GELU (Gaussian Error Linear Unit: pondera las entradas por su probabilidad bajo una distribución gaussiana suavizando su activación cerca de 0).

También realiza representaciones contextualizadas con Transformers siendo que la salida del codificador se pasa a una red que sigue la arquitectura de un Transformer, y usa una capa convolucional que actúa como un embedding posicional relativo.

Por último, Wav2Vec2 consta de un módulo de cuantización que discretiza la salida del codificador a un conjunto finito de representaciones del habla. Esta cuantización se hace con la técnica de Gumbel Softmax (Gumbel Softmax: técnica de reparameterización que aproxima muestras de una distribución categórica discreta mediante una función continua y diferenciable).

El entrenamiento del modelo se basa en un preentrenamiento con enmascaramiento en donde se toma la señal de audio y se transforma en representaciones latentes mediante un encoder. Esto se hace por bloques consecutivos de tamaño M que pueden solaparse.

Para cada paso de tiempo enmascarado, el modelo debe identificar la representación latente cuantizada correcta. Para ello, usa una pérdida contrastiva basada en la similitud del coseno.

También se realiza un fine-tuning supervisado para reconocimiento automático del habla usando CTC como función de pérdida y aplicando una versión de SpecAugment(técnica de aumento de datos diseñada específicamente para el reconocimiento automático de voz) para evitar sobreajuste.

Fuente(<https://arxiv.org/pdf/2006.11477>)

2.1.3. Random Forest

Random Forest es un algoritmo de aprendizaje automático basado en la combinación de múltiples árboles de decisión, propuesto por Leo Breiman en 2001 [?]. Perteneció a la familia de métodos *ensemble*, que mejoran el rendimiento agregando las predicciones de varios modelos más simples.

Arquitectura y funcionamiento

El algoritmo construye un “bosque” de árboles de decisión donde cada árbol se entrena de forma ligeramente diferente, introduciendo aleatoriedad controlada en dos niveles:

- **Bootstrap aggregating (bagging):** Cada árbol se entrena con una muestra aleatoria del conjunto de datos original, permitiendo repeticiones. Esto significa que algunos ejemplos aparecen varias veces en el entrenamiento de un árbol, mientras que otros (alrededor de un tercio) quedan fuera (*out-of-bag*) y sirven para validación interna.
- **Selección aleatoria de características:** En cada división de un nodo del árbol, solo se considera un subconjunto aleatorio de todas las características disponibles. Esto evita que unas pocas características muy fuertes dominen todas las decisiones.

Proceso de clasificación

Cuando llega una nueva muestra de audio con sus características (formantes, MFCC, etc.), cada árbol del bosque realiza su propia clasificación. La predicción final se decide por mayoría:

- Si la mayoría de los árboles clasifican el audio como deepfake, la predicción final es deepfake.
- Si la mayoría lo clasifica como real, la predicción final es real.

Este mecanismo de votación reduce drásticamente la varianza del modelo: aunque un árbol individual pueda cometer errores, el consenso del bosque suele ser acertado. Esto se puede ver en la siguiente imagen: 1

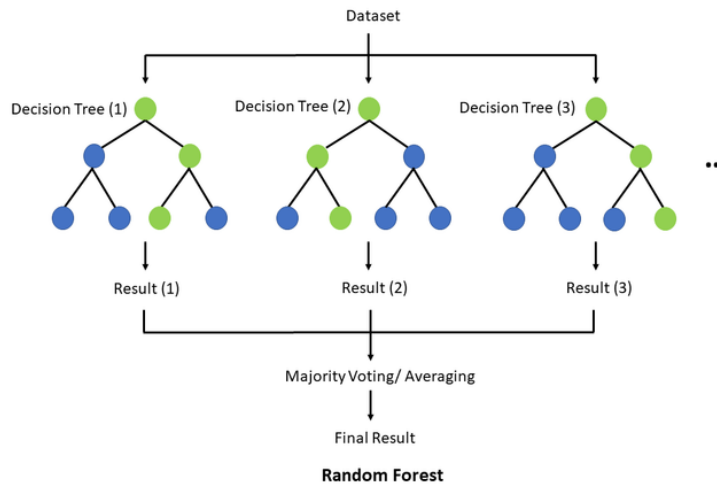


Figura 1: Voto por mayoría en el “bosque”.

Ventajas específicas para detección de deepfakes

- **Robustez ante ruido:** Las características de audio pueden contener variaciones por condiciones de grabación; Random Forest maneja bien estos datos ruidosos.
- **No requiere normalización:** A diferencia de redes neuronales o SVM, no es necesario escalar las características, lo que simplifica el pipeline.
- **Interpretabilidad:** El algoritmo proporciona una medida de importancia de cada característica, permitiendo identificar qué parámetros acústicos son más discriminativos entre voz real y sintética.
- **Resistencia al overfitting:** La combinación de bagging y aleatoriedad evita que el modelo memorice el conjunto de entrenamiento.

2.1.4. LightGBM

LightGBM (*Light Gradient Boosting Machine*) es un framework de gradient boosting desarrollado por Microsoft en 2017 [?] que optimiza el proceso de entrenamiento mediante innovaciones en la forma de construir los árboles y muestrear los datos. Mientras que Random Forest construye árboles de forma independiente y paralela, LightGBM los construye secuencialmente, donde cada nuevo árbol se especializa en corregir los errores de los anteriores.

Arquitectura y funcionamiento

El proceso de boosting funciona de la siguiente manera:

1. Se entrena un primer árbol simple que intenta clasificar las instancias según las etiquetas.
2. Se identifican las que este primer árbol clasificó incorrectamente.
3. Se entrena un segundo árbol enfocado específicamente en esos casos difíciles.
4. Se combinan ambos árboles para mejorar la precisión global.
5. El proceso se repite construyendo decenas o cientos de árboles, cada uno corrigiendo los residuos de los anteriores.

Innovaciones clave

- **Crecimiento por hoja (*leaf-wise*):** Los árboles tradicionales crecen nivel por nivel (balanceados). LightGBM, en cambio, crece expandiendo la hoja que más puede reducir el error en cada paso, lo que produce árboles más profundos en las regiones problemáticas y más eficientes en las fáciles.
- **Muestreo basado en gradientes (*GOSS*):** Durante el entrenamiento, LightGBM identifica qué instancias tienen mayor error y las prioriza. Las instancias con error pequeño se muestrean aleatoriamente para reducir el costo computacional. Esto permite entrenar con datasets grandes sin procesar toda la información redundante.
- **Agrupación de características (*EFB*):** LightGBM agrupa las características que no suelen tomar valores iguales a 0 para reducir la dimensionalidad, acelerando el entrenamiento sin pérdida de información significativa.

Proceso de clasificación

Cuando una nueva instancia llega para ser clasificada:

- Pasa secuencialmente por todos los árboles construidos.
- Cada árbol aporta una corrección o ajuste a la predicción anterior.
- La suma ponderada de todas las contribuciones produce la clasificación final (probabilidad de pertenecer a una clase).

Ventajas específicas para detección de deepfakes

- **Alta precisión:** El enfoque secuencial de corrección de errores suele superar a Random Forest en problemas complejos como la detección de deepfakes.
- **Eficiencia computacional:** El muestreo inteligente permite entrenar con grandes volúmenes de audios en menos tiempo que otros métodos de boosting.
- **Manejo de desequilibrios:** Si existen desbalances entre clases, LightGBM incorpora mecanismos para ponderar las muestras adecuadamente.

- **Early stopping:** Se puede configurar para que detenga el entrenamiento cuando añadir más árboles no mejora la validación, evitando overfitting.

2.1.5. Support Vector Machines (SVM)

Las Máquinas de Vectores de Soporte (*Support Vector Machines*, SVM) son un conjunto de algoritmos de aprendizaje supervisado desarrollados por Vladimir Vapnik y su equipo en AT&T Bell Laboratories durante la década de 1990 [?]. Originalmente diseñadas para clasificación binaria, las SVM se han convertido en uno de los métodos más robustos y teóricamente fundamentados del aprendizaje automático.

Fundamento conceptual: El hiperplano óptimo

El objetivo fundamental de una SVM es encontrar un hiperplano que separe las dos clases (en nuestro caso, voces reales y voces generadas por IA) con el máximo margen posible. Para comprender este concepto, es útil visualizar un problema simple en dos dimensiones:

- Imaginemos que representamos cada audio como un punto en un plano, donde los ejes son dos características (por ejemplo, el primer formante y el primer MFCC).
- Las voces reales se agrupan en una región del plano y las generadas por IA en otra.
- Un hiperplano en dos dimensiones es simplemente una línea recta que intenta separar ambos grupos.

La clave de SVM reside en que no cualquier línea divisora es válida: el algoritmo busca aquella que maximiza la distancia (margen) entre la línea y los puntos más cercanos de cada clase. Estos puntos críticos, que “sostienen” el margen, reciben el nombre de **vectores de soporte** (*support vectors*), de donde el algoritmo toma su nombre.

Funcionamiento del algoritmo

El proceso de entrenamiento de una SVM puede entenderse en los siguientes pasos:

1. **Identificación de vectores de soporte:** El algoritmo examina todos los puntos de entrenamiento (instancias) e identifica aquellas que son más difíciles de clasificar, es decir, las que están más cerca de la frontera entre clases.
2. **Maximización del margen:** A partir de estos vectores de soporte, la SVM calcula el hiperplano que maximiza la distancia a ambos lados. Este margen máximo proporciona la mejor capacidad de generalización: cuanto más ancho sea el margen, menor será la probabilidad de error al clasificar nuevas instancias.
3. **Clasificación de nuevas muestras:** Una vez entrenado el modelo, para clasificar una nueva instancia, simplemente se calcula a qué lado del hiperplano se sitúa.

El truco del kernel: Separando lo inseparable

Uno de los mayores desafíos en la clasificación es que las instancias rara vez son linealmente separables en el espacio original de características. Es decir, no existe una línea recta (o hiperplano) que pueda separarlas perfectamente. Para abordar esta limitación, las SVM introducen el concepto de **kernel** o núcleo.

El *truco del kernel* (*kernel trick*) funciona de la siguiente manera conceptual:

- Los datos originales (no separables linealmente) se transforman implícitamente a un espacio de mayor dimensionalidad mediante una función matemática.
- En este nuevo espacio, es posible que los datos sí sean linealmente separables.
- La magia del kernel es que realiza esta transformación sin necesidad de calcular explícitamente las coordenadas en el espacio de alta dimensión, lo que sería computacionalmente costoso.

Los kernels más comúnmente utilizados son:

- **Kernel lineal**: Asume que los datos son aproximadamente separables por un hiperplano. Útil cuando el número de características es muy grande.
- **Kernel polinomial**: Crea fronteras de decisión curvas mediante combinaciones polinómicas de las características originales.
- **Kernel RBF (*Radial Basis Function*)**: Es el más utilizado en muchos problemas. Proyecta los datos a un espacio de dimensionalidad infinita, permitiendo fronteras de decisión altamente complejas y no lineales. Se basa en la similitud entre muestras medida por distancia euclidiana:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

donde γ controla la influencia de cada muestra: un valor pequeño considera vecindarios amplios (frontera más suave), mientras que un valor grande se enfoca en vecindarios reducidos (frontera más ajustada a los datos).

Parámetros críticos

El rendimiento de una SVM depende fundamentalmente de dos parámetros:

- **C (parámetro de regularización)**: Controla el equilibrio entre tener un margen amplio y clasificar correctamente los puntos de entrenamiento. Un valor bajo de C busca un margen grande aunque se cometan algunos errores (modelo más generalista). Un valor alto de C penaliza fuertemente los errores, ajustándose más a los datos de entrenamiento (riesgo de overfitting).
- **γ (gamma)**: Específico del kernel RBF, define hasta qué punto llega la influencia de un solo punto de entrenamiento. Valores bajos significan influencia de largo alcance (frontera más suave), mientras que valores altos hacen que cada punto tenga influencia muy local (frontera más compleja).

Ventajas específicas para detección de deepfakes

- **Fundamento teórico sólido:** A diferencia de otros métodos heurísticos, SVM está basada en la teoría de optimización convexa, lo que garantiza que la solución encontrada es óptima (máximo margen global).
- **Eficaz en espacios de alta dimensión:** Las características de audio pueden generar vectores de gran dimensionalidad (decenas o cientos de coeficientes). SVM maneja bien esta situación sin colapsar por la maldición de la dimensionalidad.
- **Robustez ante overfitting:** La maximización del margen proporciona una regularización natural que mejora la generalización, crucial cuando se trabaja con datasets limitados de voces deepfake.
- **Versatilidad mediante kernels:** La capacidad de elegir diferentes kernels permite adaptar el modelo a la estructura no lineal de los datos de audio sin necesidad de transformaciones manuales.
- **Interpretabilidad limitada pero útil:** Aunque no tan interpretable como los árboles de decisión, el análisis de los vectores de soporte puede revelar qué audios son más representativos o ambiguos en la frontera de decisión.

2.1.6. AASIST

AASIST (*Audio Anti-Spoofing usign Integrated Spectro-Temporal graph attention networks*) es un modelo propuesto por Jee-weon Jung, Hee-Soo Heo, Hemlata Tak, Hye-jin Shim, Joon Son Chung, Bong-Jin Lee, Ha-Jin Yu y Nicholas Evans en 2022. La principal motivación para el desarrollo de este clasificador surge de que los artefactos que sirven para diferenciar audios falsificados de audios bona fide se suelen encontrar en intervalos temporales o espectrales. Por esto, en la práctica se usaban ensembles costosos, desde el punto de vista computacional, donde cada subsistema se centraba en ciertos artefactos. AASIST consigue reemplazar ese enfoque por un único sistema que combina información espectro-temporal buscando robustez ante diversos tipos de ataques.

Arquitectura y funcionamiento

El modelo tiene las siguientes fases:

1. Extracción de características a partir del audio crudo:
 - Se usa un **encoder** basado en RawNet2 que acaba generando un mapa de características F , $F \in \mathbb{R}^{C \times S \times T}$ siendo C , número de canales, S , número de bins espectrales y T , longitud temporal.
 - Para ello, primero, se pasa el audio por una primera capa *sinc-convolution* con 70 filtros. La diferencia frente al RawNet2 original, es que en AASIST se interpreta la salida de esta capa como una “imagen” 2D de 1 canal.
 - A continuación, esta primera representación se pasa por seis bloques residuales con **pre-activación** que van comprimiendo y transformando la información útil para tener una representación lo más compacta posible.

2. Posteriormente, se generan dos **grafos completos** que modelan, de forma independiente, los dominios temporal y espectral, siendo G_t y G_s respectivamente. A cada grafo, se le aplica “**graph pooling**” para reducir el número de nodos, conservando los más relevantes lo que permite bajar el coste computacional posterior.
3. Ambos grafos se combinan en un tercer grafo, G_{st} con $N_t + N_s$ nodos entre los que se generan todas las aristas nuevas posibles en ambas direcciones, manteniendo las aristas de los grafos anteriores.
4. Bloque HS-GAL + pooling:
 - Se aplica **atención heterogénea** usando uno de los tres vectores de proyección según la relación entre los tipos de los nodos entre los que se hace la relación.
 - Un (**stack node**), conectado de forma unidireccional a todos los nodos, que va acumulando la información espectro-temporal entre sucesivas capas HS-GAL.
 - Tras cada HS-GAL, se aplica otra vez la técnica “**graph pooling**” con el mismo objetivo anterior.
5. A continuación de crear el grafo G_{st} , se aplica la operación MGO(*Max Graph Operation*) que usa las técnicas del punto anterior. De forma paralela, se procesan dos ramas en las que se ejecutan consecutivamente dos secuencias HS-GAL-pooling. Dentro de cada rama, el “stack node” se propaga de la primera HS-GAL a la segunda. Finalmente, se combinan las salidas de ambas ramas usando un máximo elemento a elemento.
6. Por último, usando las operaciones “**max**” y “**average**” sobre cada subgrafo final(temporal y espectral), se generan 4 vectores que se combinan con el “stack node” y se pasan a la capa de salida.

Innovaciones clave

- **HS-GAL**: una capa de atención, que permite ir integrando simultaneamente ambos grafos, combinada con un “stack” node para ir acumulando información espectro-temporal
- **MGO**: un método con dos ramas aplicando paralelamente HS-GAL más una fase de “pooling” lo que permite que cada secuencia pueda aprender distintos grupos de artefactos.
- **Nuevo esquema de readout**: permite concatenar la información contenida en el “stack node” con las estadísticas sacadas de la operaciones “max” y “average” finales.

Ventajas específicas para detección de deepfakes

- **Integración espectro-temporal:** al modelar conjuntamente los grafos temporal y espectral en un grafo heterogéneo, permite encontrar información cruzada que se podría perder si se tratara por separado.
- **Stack node:** esta técnica “acumulativa” ayuda a encontrar artefactos de spoofing cuando no están presentes en un solo nodo y se encuentran distribuidos en varios instantes o bandas.
- **Readout:** al juntar estadísticas globales(average) con evidencia puntual fuerte(max), facilita la detección de diferentes patrones de “spoofing”.
- **Ramas paralelas:** en MGO, se introducen dos ramas procesando en paralelo, permitiendo captar pistas complementarias.

2.2. Modelos de explicación

2.2.1. SHAP

Basado en la teoría de juegos cooperativos, SHAP asigna a cada característica acústica un valor de importancia que representa su contribución marginal. El valor de Shapley ϕ_i para una característica i se calcula como:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$$

Este método es preferido en el análisis forense debido a su consistencia matemática y su capacidad para ofrecer tanto explicaciones locales como una visión global de la importancia de los descriptores (como los coeficientes MFCC o el pitch).

2.2.2. LIME

LIME aproxima localmente la superficie de decisión de un modelo de caja negra mediante un modelo lineal interpretable. En el dominio del audio, el proceso se define mediante la perturbación de la entrada x (espectrogramas o marcos temporales) para observar las variaciones en la salida $f(x)$. La explicación g se obtiene minimizando:

$$\xi(x) = \arg \min_{g \in G} L(f, g, \pi_x) + \Omega(g)$$

donde L es la pérdida de fidelidad, π_x define la vecindad de la muestra y $\Omega(g)$ mide la complejidad del modelo sustituto.

2.2.3. Grad-CAM

Utiliza los gradientes de la última capa convolucional para producir un mapa de calor que resalta las regiones del espectrograma que influyen en la decisión. Aunque es eficaz para detectar sesgos en el conjunto de datos, su baja resolución espacial a menudo dificulta la identificación de discontinuidades de fase de micro-segundos, típicas de los vocoders neuronales.

2.2.4. Integrated Gradients

IG aborda la saturación del gradiente integrando las variaciones a lo largo de una trayectoria desde una entrada de referencia (baseline) x' hasta la entrada real x . La atribución para la característica i se define como:

$$IG_i(x) = (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial f(x' + \alpha(x - x'))}{\partial x_i} d\alpha$$

En audio, la selección de la línea base (silencio vs. ruido blanco) es un factor crítico para evitar explicaciones espurias.

2.2.5. DICE

Modelos como DiCE permiten generar ejemplos sintéticos mínimos que cambiarían la predicción del modelo. Esto es vital para analistas forenses, ya que permite identificar qué características de "naturalidad" le faltan a una muestra para ser considerada auténtica, proporcionando una comprensión causal del fenómeno de los deepfakes.

2.2.6. SincNet

SincNet sustituye las capas de convolución estándar por filtros de banda basados en la función Sinc. El modelo solo aprende las frecuencias de corte f_1 y f_2 , permitiendo una interpretación física directa de los rangos espectrales (como formantes o armónicos) que el sistema utiliza para discriminar la voz real de la sintética.

2.2.7. ProtoPNet

Estos modelos clasifican la señal encontrando similitudes con "prototipos" aprendidos durante el entrenamiento. Ante un audio sospechoso, el modelo proporciona una explicación del tipo: "Este segmento se clasifica como deepfake porque se asemeja a este patrón de artefacto espectral observado en el modelo de síntesis WaveNet".

3. Metodología

3.1. Dataset

Para encontrar un dataset público que tuviera voces tanto reales como clonadas y que además fueran por pares (i.e. un audio de voz real cuya transcripción es "Hoy he comido en un restaurante chino" y un audio clonando dicha voz diciendo lo mismo) ha sido complicado. Hemos echado un ojo a bastantes datasets. Vamos a comentar los que hemos tenido en cuenta.

1. HABLA [?]: Este dataset cuenta con una gran cantidad de audios tanto reales, como generados (con técnicas como TTS) y voces clonadas (con técnicas como Cycle-GAN). Lo hemos descartado para una primera iteración del proyecto, pero no lo descartamos para una próxima iteración.

2. audioMNIST [?]: Este dataset cuenta con 30 mil audios de los números del 0 al 9 en inglés. Lo hemos descartado porque todos los audios son voces reales, por lo que no nos sirve para nuestra tarea.
3. ASVSpooof2021 [?]: Este dataset forma parte de una serie de desafíos internacionales organizados por la comunidad de verificación automática de audio cuyo objetivo es evaluar y mejorar los sistemas capaces de detectar voz manipulada. El dataset se compone de tres grupos de audio, todos ellos con voz real y generada mezclada, logical access con grabaciones transmitidas mediante canales de voz con efectos de codificación y transmisión, physical access con grabaciones de voz auténticas y ataques de reproducción en espacios físicos reales y speech deepfake con grabaciones generadas sintéticamente y comprimidas de formas distintas. Este dataset lo hemos descartado por no contar con pares de audio aunque se pueden rescatar algunos audios en inglés sobre todo para probar los modelos que probemos más adelante.
4. Kaggle
5. Wavefake []: Este dataset de Kaggle cuenta con gran cantidad de audios generados. Como tal, nos sirvió para probar el primer prototipo de extracción de características y buscar librerías como librosa o speechRecognition. Este dataset, al no tener los audios reales, no nos sirve para luego desarrollar los modelos.

El dataset por el que nos hemos decantado se llama **ODSS**. Hemos usado este conjunto de audios porque ya tiene creado un par para cada audio, lo que nos ahorra tiempo de generar nuestros propios audios. A esta ventaja se le suma que ODSS ya contenía audios en Inglés y en Español, ambos idiomas son sumamente hablados por lo que es interesante centrarse en ellos.

Con todo esto mencionado, hemos tomado un subset de 100 audios de cada idioma. La mitad son audios naturales y la otra mitad audios generados. Más adelante, probaremos a usar más cantidad de audios y compararemos el rendimiento.

3.2. Extracción de características

Para poder clasificar los audios en reales y generados tenemos que pensar primero en qué datos necesitamos pasar a los modelos para que aprendan. Podemos separar los datos que debemos proporcionar a dichos modelos en 3 grupos:

1. Características. Podemos extraer características de la voz como el tono fundamental, los formantes, las pausas, y otros. Con estos datos, los modelos pueden aprender las relaciones entre los dos grupos de audios.
2. Gráficas. Podemos extraer gráficas de la voz como la forma de onda, el espectrograma de MEL, y otros. Con estas gráficas las CNN pueden aprender los puntos clave que diferencian a los audios reales de los generados.
3. Audio crudo. Existen modelos llamados wav2vec los cuales aprenden directamente de los audios, sacando un vector denso de características latentes en los

audios, para después hacer ajuste fino o *fine-tuning* de un modelo para clasificar los audios.

Vamos a ver más en detalle qué características de cada tipo sacamos.

Características

Podemos extraer características de la voz como el tono fundamental, los formantes, las pausas, y otros. Con estos datos, los modelos pueden aprender las relaciones entre los dos grupos de audios. Tenemos que destacar que las características de este tipo extraídas son mayormente arrays en el que cada componente es un valor en un momento determinado. Esto es porque la voz tiene la componente temporal, por lo que hay que dividir cada audio en marcos temporales (frames). Además, los modelos que aplicaremos no entrenan con arrays, por lo que los promediaremos con la media y/o desviación típica. Vamos a explicar cada característica que extraemos de los audios:

1. **Valor cuadrático medio (rms):** Mide la energía promedio de una señal acústica (i.e. la intensidad o volumen general del audio). Se extrae dividiendo la señal en frames, para cada frame se calcula la raíz cuadrada del promedio de los valores de onda al cuadrado y, por último, se promedian esos valores. En voz humana natural, el RMS tiende a tener variaciones más orgánicas, mientras que audios sintéticos pueden mostrar patrones más uniformes.
2. **Tasa de cruces por cero (zcr):** Mide el número de veces que la señal cambia de signo por segundo, lo que nos permite distinguir entre los sonidos tonales (periódicos) y los ruidosos (no periódicos). Se extrae detectando los puntos donde la señal cambia de signo, se cuentan dichos puntos por unidad de tiempo y se normaliza por la longitud del frame. Los sonidos sordos (como /s/, /f/) tienen ZCR alto, mientras los sonoros (/a/, /m/) tienen ZCR bajo.
3. **Centroide espectral:** Mide el centro de masa del espectro de frecuencias, lo que nos dice, de forma ponderada, si la energía de un sonido se concentra en las frecuencias graves (bajas) o en las frecuencias agudas (altas). Se obtiene calculando la Transformada Rápida de Fourier (FFT) para cada frame para obtener el espectro de frecuencias, se ponderan las frecuencias por su magnitud (energía de dicha frecuencia) y se calcula la media de estas. Cabe mencionar que las voces femeninas tienden a tener centroides más altos que masculinas.
4. **Ancho de banda espectral:** Mide qué tan dispersas están las frecuencias alrededor del centroide, para ver si el sonido es “puro” como un silbido o una sílaba mantenida, o es “ruidoso” como el ruido blanco o un aplauso. Se extrae hallando el centroide como antes y se calcula la desviación típica ponderada en vez de la media ponderada.
5. **Punto de caída espectral (spectral rolloff):** Mide la frecuencia por debajo de la cual se concentra un determinado porcentaje de la energía total del espectro, es decir, la frecuencia límite para un porcentaje de energía. Normalmente, dicho porcentaje es del 85 %, aunque puede variar entre 75 % a 90 %, y funciona como un percentil. Se obtiene calculando el centroide igual que antes y sumando la

energía progresivamente hasta que lleguemos a una frecuencia con la que nos pasamos de dicho porcentaje. Es útil para distinguir entre sonidos con energía concentrada en bajas frecuencias, en un amplio espectro, o en altas frecuencias.

6. **Planitud espectral:** Mide cómo de plano es el sonido. Varía entre 0 y 1 siendo 0 un sonido con picos pronunciados y 1 un sonido con pocos picos. Se obtiene calculando el espectro como antes (FFT) y dividiendo la media geométrica del espectro entre la media aritmética del mismo.
7. **Tono fundamental (F0):** Es la frecuencia más baja y periódica de un sonido. En el contexto de la voz, es la frecuencia a la que vibran tus cuerdas vocales. El tono fundamental puede verse en, por ejemplo, las emociones, las preguntas frente a afirmaciones, las notas musicales, entre otras, por lo que nos da mucha información acerca de algo o alguien. Hay varias formas de calcularlo, y ninguna es 100 % precisa. Nosotros nos decantamos por la función PYIN, la cual primero obtiene las frecuencias fundamentales mediante el algoritmo YIN y después aplica el algoritmo probabilístico de Viterbi para obtener la cadena F0 más probable. Estos son algoritmos más sofisticados que combinan autocorrelación, análisis espectral y probabilidades para evitar errores comunes como la octava errónea (detectar 200Hz por ruido en vez del tono real de 100Hz), zonas no voiceadas (unvoiced) y ruido de fondo.
8. **Formantes:** Son las frecuencias de resonancia del tracto vocal (la boca, la garganta y la nariz). Estas frecuencias se amplifican de forma natural cuando hablamos, dependiendo de la forma que le demos a la boca.
 - a) F1 (Primer formante): Relacionado con la abertura de la mandíbula (qué tan abierta está la boca).
 - b) F2 (Segundo formante): Relacionado con la posición de la lengua (adelante o atrás).
 - c) F3 (Tercer formante): Relacionado con la posición de la punta de la lengua y tiene mucha importancia en las vocales raras en la identidad del hablante.

Para extraerlos hay varias alternativas, nosotros usamos el método de Burg para estimar las formantes, después se muestrean 100 puntos a lo largo del audio y se calcula la media.

9. **Relación Armónico-Ruido:** Mide la proporción entre energía periódica (armónicos) y energía aperiódica (ruido). Esta relación es una indicadora directa de la eficiencia fonatoria y la salud vocal: puede indicar la existencia de patologías como voz ronca, o afonía; una fatiga vocal; o, en algunos casos, emociones. Hay varias formas de calcularla: como el método de autocorrelación o el de análisis espectral. Nosotros usamos la función `to_harmonicity` de `parselmout.Sound`, la cual la calcula con el método de autocorrelación: primero halla el F0 de los frames; después aplica la función de autocorrelación normalizada $r(\tau) = \frac{\int x(t) \cdot x(t+\tau) dt}{\int x^2(t) dt}$; luego busca el máximo de la función de autocorrelación en el intervalo alrededor del período fundamental esperado, este máximo corresponde a la correlación

entre la señal y su versión desplazada un período; tras esto calcula el hnr en decibelios $HNR = 10 \cdot \log_{10} \left(\frac{r_{\max}}{1-r_{\max}} \right)$ y, por último, aplica interpolación parabólica para obtener valores más precisos del máximo y realiza correcciones para compensar el efecto de la ventana de análisis. Es interesante saber que las voces humanas naturales suelen oscilar entre los 10 a 25 dB, por lo que unos valores extremos pueden indicar voz sintética.

10. **Proporción de silencio:** Mide el porcentaje del tiempo o de frames donde no hay voz activa, es decir, hay silencio o ruido de fondo hasta cierto umbral. La forma de calcularla es muy simple: se establece un umbral de energía el cual por debajo una energía es considerada silencio, se calcula el rms y se le aplica el filtro del umbral, finalmente se divide el número de frames considerados silencio entre el número total de frames. Esto, pese a su simpleza, puede ser determinante en el análisis de una voz ya que las IAs pueden generar patrones de pausas poco naturales o demasiado regulares.
11. **Coefficientes Cepstrales en Frecuencia de Mel (mfcc):** Son una representación compacta del espectro de frecuencias que imita cómo percibe el sonido el oído humano. En lugar de trabajar con frecuencias lineales (Hz), trabajan con una escala llamada escala Mel, que es más sensible a los cambios en frecuencias bajas que en altas (como nuestro oído). Son un total de 13 coeficientes: el primero representa la energía promedio del espectro (similar al RMS, pero en el dominio cepstral); mientras que el resto representan la forma detallada del espectro: los picos, los valles, las pendientes. Cada coeficiente añade un nivel de detalle diferente. Estos coeficientes son muy importantes ya que son, con diferencia, la característica más utilizada en: reconocimiento de voz (Speech-To-Text), reconocimiento de hablante (Identificación), clasificación de géneros musicales, entre muchas otras aplicaciones. Hoy en día son un estándar en reconocimiento de voz. Se extraen mediante un proceso más laborioso: primero se hace un pre-énfasis, amplificando las frecuencias altas, ya que estas tienen poca energía; después se calcula el espectro de frecuencias de cada frame; tras esto se aplica un banco de filtros triangulares espaciados según la escala Mel, donde por debajo de los 1000 Hz se aplican filtros lineales (nuestro oído distingue bien los cambios en frecuencias bajas) y por encima de los 1000 Hz se aplican filtros logarítmicos (nuestro oído distingue peor los cambios en frecuencias altas), con la fórmula $Mel(f) = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right)$ y, para terminar, aplica la Transformada Coseno Discreta (DCT), la cual decorrelaciona las bandas de Mel, que suelen estar correlacionadas entre sí, compacta la información y devuelve los 13 primeros coeficientes (pueden aparecer entre 20 y 40, pero no son muy representativos).
12. **Asimetría (skewness):** Mide hacia qué lado se inclina la distribución de la energía en un conjunto de datos. Si hay una asimetría positiva, la energía se concentra en valores bajos, con una cola larga hacia los valores altos. Si hay una asimetría negativa, la energía se concentra en valores altos, con una cola larga hacia los valores bajos. Si no hay asimetría, la energía está equilibrada alrededor del centro (como una campana de Gauss). Se calcula restando la media a cada muestra

y elevar al cubo, luego hallando el promedio de estas desviaciones cúbicas, y, finalmente, dividiendo por la desviación estándar al cubo para estandarizar. Cabe mencionar que unas distribuciones de amplitud asimétricas pueden indicar características anómalas en voces sintéticas.

13. **Curtosis:** Mide qué tan concentrados están los valores alrededor de la media y qué tan pesadas son las colas de la distribución. En audio, la curtosis nos dice qué tan "espijado" o "suave" es un sonido en términos de cómo se distribuyen sus amplitudes (en el tiempo) o su energía (en la frecuencia). Se obtiene con la siguiente fórmula
$$\text{Curtosis} = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^4}{\left(\sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2} \right)^4}$$
. Adicionalmente, se puede restar

3 a la curtosis para que la mitad (mesocúrtica) se halle en 0. Podemos destacar que una curtosis alta (distribución picuda) puede aparecer en audios con compresión excesiva o artefactos de síntesis.

14. **Transcripción:** Para la transcripción de los audios usamos modelos de **vosk**. Esta elección se debe a que tiene modelos para ambos idiomas que vamos a usar y además, 2 modelos dentro de cada lenguaje. El que usamos para las pruebas es el small ya que es más rápido y por lo tanto más útil para esta primera fase pero dejamos en consideración el modelo full para futuras fases ya que tiene mayor precisión. Sobre esto, cabe mencionar que primero probamos el modelo whisper pero tras varias pruebas con errores y la incapacidad para hacer funcionar, lo descartamos y es cuando escogimos vosk para esta tarea. Es interesante tener en cuenta la transcripción de un audio ya que en ocasiones, si no podemos sacar texto en claro, pero se sabe que hay voces, es muy probable que el audio sea generado.

3.3. Preprocesamiento

3.4. Modelos de clasificación

3.4.1. Perceptrón multicapa

El motivo de probar un MLP para esta tarea de clasificación fue sobre todo por explorar que posibles opciones tenemos a la hora de realizar esta tarea.

Hemos usado el código de extracción de características para procesar los audios que tenemos en el dataset personalizado con 100 audios en español (50 reales y 50 generados a partir de esos reales) y sacar sus características.

Una vez preprocesados los audios con sus características extraídas, empleamos una CNN (red neuronal convolucional) para sacar un embedding de 128 dimensiones del espectrograma del audio y un MLP (perceptrón multicapa) para sacar un embedding de 64 dimensiones de las características paralingüísticas del audio. Tras esto, las concatenamos teniendo un embedding de 192 dimensiones siendo las primeras 64 las características y las otras 128 las del espectrograma.

Después, creamos un perceptrón multicapa cuyo objetivo es clasificar el audio en si es

generado o es real empleando para ello lógica difusa (*fuzzy logic*). Para ello, pasamos la salida del perceptrón por un controlador fuzzy que irá aprendiendo también sus parámetros y adaptará la salida para que esté entre 0 y 1 siendo que cuanto más cercano al 0, más posible es que el audio sea real y no sea generado. En la práctica, como el dataset es de 100 datos, no hay suficientes para entrenar un perceptrón de forma correcta ya que cuando hicimos las pruebas, todas las predicciones del perceptrón no se alejan del 0.5 que en este contexto de probabilidad continuo significa que el modelo está indeciso y no lo sabe clasificar como real o como generado.

El flujo de datos de este método se resume en coger los archivos de audios, pasarlos por el extractor de características para obtener todas sus características y agruparlo todo en el mismo dataframe. Después pasamos todas las características del audio por un MLP para sacar un embedding de 64 dimensiones. También sacamos el espectrograma para pasarlo por una pequeña CNN y sacar otro embedding de 128 dimensiones. Ambos embeddings los concatenamos en uno de 192 dimensiones que es lo que se le pasa al MLP final para calcular la pertenencia de los audios a las clases real o generado.

Los resultados obtenidos por el modelo muestran un gran sobreajuste dado los pocos audios de los que disponemos en el dataset que tenemos.

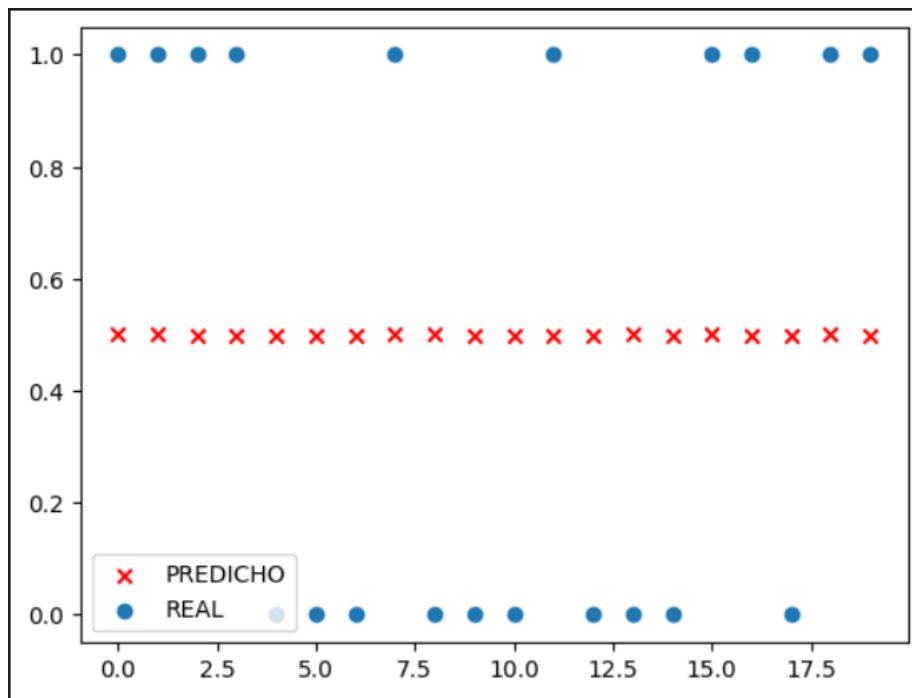


Figura 2: Resultados del MLP.

Las conclusiones que podemos sacar sobre este modelo a la vista de los resultados

son que para crear un modelo de clasificación desde 0 y que sea eficaz en su labor, necesitaríamos muchísimos más audios de los que disponemos, además, en lo que se refiere a la posterior tarea de aplicar elementos de IA explicable al modelo, al usar embeddings como entrada, no podemos aplicarle directamente ninguna de esas técnicas.

3.4.2. Fine-tuning Wav2Vec2

El propósito de realizar un fine-tuning a este modelo para adaptarlo a nuestra tarea se basa en que al tener un dataset de audios muy reducidos, necesitamos partir de un modelo ya entrenado con audios, aunque no fuera para la clasificación de los mismos, ya que así podemos usar nuestros audios para reentrenar el modelo para clasificación.

El modelo que elegimos fue Wav2Vec2, que es un modelo de aprendizaje profundo para procesamiento de voz que permite obtener representaciones acústicas de alta calidad directamente desde la señal cruda de audio, sin necesidad de características manuales. Fue propuesto por Meta en 2020.

Este modelo recibe como entrada la forma de onda sin procesar del audio (audio crudo) y su salida debe ser decodificada utilizando Wav2Vec2CTCTokenizer.

El fine-tuning se hará con el dataset de 100 datos que tenemos para entrenar el Wav2Vec2 en la tarea de clasificación de audio. Al igual que con el modelo anterior, también lo pasaré por un controlador fuzzy para que nos diga cuan probable es que el audio sea real o IA.

Una vez tenemos el dataset, hay que preprocesar los audios ya que en un inicio solo tiene las rutas de los audios por lo que ahora hay que cargar los audios de verdad. Para esta tarea usaremos el propio extractor de características que tiene el modelo (Wav2Vec2FeatureExtractor) fijando la frecuencia de muestreo a 16KHz y activando el padding que rellenará los audios cortos con ceros y trucará los audios largos para que todos tengan la misma longitud. Este preprocesado resultará en que ahora en el dataset, en lugar de haber rutas a los audios, hay tensores de Torch representándolos.

Ahora que ya tenemos el dataset de entrenamiento preprocesado, lo volvemos a dividir en entrenamiento y validación y entrenamos el modelo ya preentrenado para la nueva tarea de detección de voz generada por . Para evaluar el modelo se emplearon tres métricas, la precisión (verdaderos positivos / (verdaderos positivos + falsos positivos), la f1 (Media armónica entre precisión y recall) y el AUC (Área bajo la curva: Capacidad del modelo para distinguir entre clases positivas y negativas). El fine-tuning ha sido muy leve ya que con los 90 audios (80 de entrenamiento y 10 de validación) hay que congelar el aprendizaje del modelo para evitar un sobreentrenamiento.

Una vez hecho el fine-tuning, hacemos el mismo preprocesado para el dataset de test y ponemos el modelo en modo evaluación. Los resultados que de el modelo se pasan por un clasificador fuzzy que se encargará del postprocesado de la salida que la normalizará entre 0 y 1 siendo que los audios cuya score está entre 0.4 y 0.6 no los sabe clasificar, los que están por debajo de 0.4 son audios reales y los que están por encima de 0.6

son audios generados. Este postprocesado también puede hacer que el modelo tome decisiones más polarizadas obligándolo a mojarse.

Este modelo muestra unos resultados muy buenos ya que tanto la precisión, como el f1-score y el auc rondan siempre el 0,9. Si que es cierto que los fallos se concentran sobre todo en los audios reales diciendo que son generados.

```
{'eval_loss': 0.43612009286880493,
 'eval_model_preparation_time': 0.008,
 'eval_accuracy': 0.9,
 'eval_f1': 0.9090909090909091,
 'eval_auc': 0.92,
 'eval_runtime': 36.3305,
 'eval_samples_per_second': 0.551,
 'eval_steps_per_second': 0.138}
```

Figura 3: Métricas fine-tuning Wav2Vec2.

	# audio_id	# score	⚠ decision_fuzzy	# label
	0	0	0.37823838	posiblemente real
	1	1	0.71498334	posiblemente IA
	2	2	0.33863166	posiblemente real
	3	3	0.71035093	posiblemente IA
	4	4	0.71441114	posiblemente IA
	5	5	0.6257453	posiblemente IA
	6	6	0.3466234	posiblemente real
	7	7	0.31925088	posiblemente real
	8	8	0.72027296	posiblemente IA
	9	9	0.70500165	posiblemente IA

Figura 4: Resultados fine-tuning Wav2Vec2.

Sobre esto errores que comete el modelo intentamos integrated gradients para ver una explicación de por qué el modelo fallaba. Lamentablemente, este modelo como entrada recibe embeddings de los audios por lo que no se le pueden aplicar técnicas de IA explicable directamente.

3.4.3. Random Forest

Como hemos comentado sobre los Random Forest, estos construyen un conjunto o "bosque" de árboles de decisión donde cada árbol se entrena de forma ligeramente diferente, introduciendo aleatoriedad controlada.

Justificación del modelo

Se seleccionó Random Forest como uno de los clasificadores base debido a su robustez frente al sobreajuste, su capacidad para manejar características heterogéneas y su interpretabilidad mediante el análisis de importancia de variables.

Configuración experimental

El proceso experimental para Random Forest se estructuró en tres fases: (1) establecimiento de un modelo baseline, (2) validación cruzada para evaluar robustez, (3) optimización de hiperparámetros mediante búsqueda sistemática.

División de datos El conjunto de datos se particionó en entrenamiento (80 %) y prueba (20 %) utilizando muestreo estratificado para mantener la proporción de clases en ambos conjuntos. Esta estratificación es crucial cuando existe desbalance entre voces reales y generadas.

Listing 1: División estratificada de datos

```
X_train , X_test , y_train , y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=12,
    stratify=y # Mantiene proporcion de clases
)
```

Modelo baseline Se estableció una configuración inicial con hiperparámetros por defecto para obtener una referencia de rendimiento:

Listing 2: Configuración del modelo baseline Random Forest

```
rf_baseline = RandomForestClassifier(
    n_estimators=100,      # Numero de arboles
    max_depth=10,         # Profundidad maxima
    min_samples_split=5,  # Minimo muestras para dividir nodo
    min_samples_leaf=2,   # Minimo muestras en hoja
    max_features='sqrt',  # Caracteristicas por division
    random_state=12,
    n_jobs=-1,            # Usar todos los cores
    class_weight='balanced' # Ajuste para clases desbalanceadas
)
```

El parámetro `class_weight='balanced'` ajusta los pesos de las clases de forma inversamente proporcional a su frecuencia, compensando posibles desbalances en el conjunto de datos.

Validación cruzada

Para evaluar la robustez del modelo y evitar que los resultados dependan de una única partición de los datos, se realizó validación cruzada estratificada con 5 folds:

Listing 3: Validación cruzada con Random Forest

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=12)

cv_scores_accuracy = cross_val_score(rf_baseline, X_train, y_train,
                                     cv=cv, scoring='accuracy')
cv_scores_f1 = cross_val_score(rf_baseline, X_train, y_train,
                              cv=cv, scoring='f1')
cv_scores_auc = cross_val_score(rf_baseline, X_train, y_train,
                              cv=cv, scoring='roc_auc')
```

La validación cruzada proporciona una estimación más realista del rendimiento esperado en datos no vistos, reportándose la media y la desviación estándar de las métricas.

Optimización de hiperparámetros

Se realizó una búsqueda sistemática de hiperparámetros mediante GridSearchCV con validación cruzada de 5 folds, optimizando el F1-score. El espacio de búsqueda incluyó:

Listing 4: Grid Search para Random Forest

```
param_grid = {
    'n_estimators': [50, 100, 200],      # Numero de arboles en el bosque
    'max_depth': [5, 10, 15, None],      # Profundidad maxima de cada arbol
    'min_samples_split': [2, 5, 10],     # Minimo de muestras para dividir un nod
    'min_samples_leaf': [1, 2, 4],       # Minimo de muestras en una hoja
    'max_features': ['sqrt', 'log2']     # Numero de caracteristicas a considerar
}

grid_search = GridSearchCV(
    RandomForestClassifier(random_state=12, class_weight='balanced'),
    param_grid,
    cv=cv,
    scoring='f1', # Optimizacion de F1-score
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)
rf_optimized = grid_search.best_estimator_
```

Métricas de evaluación

Para una evaluación exhaustiva de los modelos, se emplearon las siguientes métricas:

- **Accuracy (Exactitud):** Proporción de predicciones correctas sobre el total.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision (Precisión):** Proporción de predicciones positivas correctas.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensibilidad):** Proporción de positivos reales correctamente identificados.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** Media armónica de precisión y recall.

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **AUC-ROC** (Área bajo la curva ROC): Mide la capacidad del modelo para distinguir entre clases, independientemente del umbral de clasificación. Un valor de 0.5 indica clasificación aleatoria, mientras que 1.0 indica clasificación perfecta.
- **MCC** (Coeficiente de correlación de Matthews): Métrica equilibrada que considera todos los elementos de la matriz de confusión, especialmente útil en problemas con clases desbalanceadas. Su rango es $[-1, 1]$, donde 1 indica predicción perfecta, 0 predicción aleatoria y -1 predicción inversa perfecta.

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

La función de evaluación implementada automatiza el cálculo de todas estas métricas y genera visualizaciones:

Listing 5: Evaluación de un modelo

```
def evaluacion_modelo(model_name, y_true, y_pred, y_pred_proba):  
    """  
    Evalua un modelo con metricas completas y visualizaciones.  
    """  
    # Calculo de metricas  
    accuracy = accuracy_score(y_true, y_pred)  
    auc = roc_auc_score(y_true, y_pred_proba)  
    f1 = f1_score(y_true, y_pred)
```



```

mcc = matthews_corrcoef(y_true , y_pred)
precision = precision_score(y_true , y_pred)
recall = recall_score(y_true , y_pred)

# Matriz de confusion
cm = confusion_matrix(y_true , y_pred)

# Visualizaciones (matriz de confusion , curva ROC)
# ...

return {
    'accuracy': accuracy , 'precision': precision ,
    'recall': recall , 'f1': f1 ,
    'auc': auc , 'mcc': mcc ,
    'confusion_matrix': cm
}

```

3.4.4. LightGBM

Como hemos comentado antes, a diferencia de Random Forest, que construye árboles de forma independiente y paralela, LightGBM los construye secuencialmente, donde cada nuevo árbol se especializa en corregir los errores de los anteriores.

Justificación del modelo

Hemos usado LightGBM debido a que presenta ventajas significativas para la tarea de detección de deepfakes de audio, como pueden ser su eficiencia computacional para grandes volúmenes de datos o su regulación integrada con los parámetros L1 y L2.

Configuración experimental

El proceso experimental para LightGBM se estructuró en cuatro fases: (1) establecimiento de un modelo baseline, (2) validación cruzada para evaluar robustez, (3) optimización de hiperparámetros mediante búsqueda sistemática, y (4) evaluación comparativa con métricas exhaustivas.

Preparación de datos A diferencia de otros algoritmos, LightGBM puede manejar directamente datos sin normalización, aunque requiere que no existan valores nulos.

Modelo baseline con LightGBM Se estableció una configuración inicial utilizando la interfaz nativa de LightGBM, que ofrece un control más fino sobre el proceso de entrenamiento:

Listing 6: Configuración del modelo baseline LightGBM

```
# Creacion de datasets optimizados de LightGBM
```

```

train_data = lgb.Dataset(X_train, label=y_train)
test_data = lgb.Dataset(X_test, label=y_test, reference=train_data)

# Parametros baseline
params_baseline = {
    'objective': 'binary',
    'metric': 'binary_logloss',
    'boosting_type': 'gbdt',
    'num_leaves': 31,
    'learning_rate': 0.05,
    'feature_fraction': 0.9,
    'bagging_fraction': 0.8,
    'bagging_freq': 5,
    'verbose': 0,
    'random_state': 12,
    'is_unbalance': True,
    'num_threads': -1
}

# Clasificacion binaria
# Metrica de evaluacion
# Gradient Boosting Decision Tree
# Numero maximo de hojas
# Tasa de aprendizaje (eta)
# Fraccion de caracteristicas por iteracion
# Fraccion de datos por iteracion
# Frecuencia de bagging
# Modo silencioso

# Compensacion de clases desbalanceadas
# Usar todos los cores

# Entrenamiento con early stopping
model_baseline = lgb.train(
    params_baseline,
    train_data,
    valid_sets=[test_data],
    num_boost_round=1000,
    callbacks=[
        lgb.early_stopping(50), # Detiene si no mejora en 50 rondas
        lgb.log_evaluation(100) # Log cada 100 iteraciones
    ]
)

```

El parámetro `is_unbalance=True` activa la compensación automática para clases desbalanceadas, ajustando los pesos de forma inversamente proporcional a la frecuencia de cada clase.

Early stopping Una ventaja importante de LightGBM es la capacidad de detener el entrenamiento cuando añadir más árboles no mejora el rendimiento en validación. Esto:

- Previene el sobreajuste (overfitting)
- Ahorra tiempo computacional
- Determina automáticamente el número óptimo de iteraciones

Validación cruzada

Para LightGBM se utilizó la interfaz compatible con scikit-learn (LGBMClassifier) que permite integrarse con las herramientas de validación cruzada estándar:

Listing 7: Validación cruzada con LightGBM

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

# Usamos la interfaz scikit-learn de LightGBM
lgb_sklearn = lgb.LGBMClassifier(
    objective='binary',
    boosting_type='gbdt',
    num_leaves=31,
    learning_rate=0.05,
    random_state=12,
    is_unbalance=True,
    n_jobs=-1
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=12)

cv_scores_accuracy = cross_val_score(lgb_sklearn, X_train, y_train,
                                     cv=cv, scoring='accuracy')
cv_scores_f1 = cross_val_score(lgb_sklearn, X_train, y_train,
                               cv=cv, scoring='f1')
cv_scores_auc = cross_val_score(lgb_sklearn, X_train, y_train,
                                cv=cv, scoring='roc_auc')
```

La validación cruzada proporciona una estimación más robusta del rendimiento, reportándose la media y la desviación estándar de las métricas.

Optimización de hiperparámetros

LightGBM cuenta con un conjunto de hiperparámetros específicos, diferentes a los de Random Forest. La búsqueda se centró en aquellos que más influyen en el rendimiento:

- **num_leaves**: Controla la complejidad del modelo. Valores altos permiten capturar patrones más complejos pero aumentan el riesgo de sobreajuste.
- **learning_rate**: Tasa de aprendizaje (eta). Valores pequeños requieren más iteraciones pero pueden lograr mejor convergencia.
- **n_estimators**: Número de iteraciones de boosting.
- **min_child_samples**: Mínimo de muestras requeridas en una hoja. Controla la profundidad y evita hojas con pocas muestras.
- **subsample** y **colsample_bytree**: Fracciones de muestras y características para cada árbol, introduciendo aleatoriedad.

- **reg_alpha** y **reg_lambda**: Regularización L1 y L2 para controlar la complejidad.

Listing 8: Grid Search para LightGBM

```
param_grid = {
    'num_leaves': [15, 31, 63],           # Complejidad del modelo
    'learning_rate': [0.01, 0.05, 0.1],   # Tasa de aprendizaje
    'n_estimators': [100, 200, 300],       # Numero de iteraciones
    'min_child_samples': [20, 50, 100],    # Minimo muestras por hoja
    'subsample': [0.6, 0.8, 1.0],         # Fraccion de muestras
    'colsample_bytree': [0.6, 0.8, 1.0],   # Fraccion de caracteristicas
    'reg_alpha': [0, 0.1, 0.5],          # Regularizacion L1
    'reg_lambda': [0, 0.1, 0.5]           # Regularizacion L2
}

grid_search = GridSearchCV(
    lgb.LGBMClassifier(objective='binary', random_state=42,
                        is_unbalance=True, n_jobs=-1),
    param_grid,
    cv=cv,
    scoring='f1',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)
lgb_optimized = grid_search.best_estimator_
```

Métricas de evaluación

Se emplearon las mismas métricas que en Random Forest para mantener la comparabilidad:

- **Accuracy**: Proporción de predicciones correctas.
- **Precision**: Proporción de predicciones positivas correctas.
- **Recall**: Proporción de positivos reales correctamente identificados.
- **F1-Score**: Media armónica de precisión y recall.
- **AUC-ROC**: Área bajo la curva ROC.
- **MCC**: Coeficiente de correlación de Matthews.

La función de evaluación implementada es la misma que para Random Forest.

3.4.5. Support Vector Machine (SVM)

Como ya hemos visto sobre los SVM, estos se fundamentan en la búsqueda del hiperplano óptimo que maximiza el margen de separación entre clases, lo que proporciona una base teórica sólida y garantías de generalización.

Justificación del modelo

Usamos SVM ya que presenta características especialmente relevantes para la detección de deepfakes de audio que hemos comentado antes, como la eficacia en espacios de alta dimensión (como puede ser el mundo del lenguaje natural) o su fundamento teórico robusto.

Importancia de la normalización

Una característica crítica de SVM es su sensibilidad a la escala de las características. Las variables de audio presentan escalas muy diferentes:

- **Formantes:** Del orden de cientos de Hz (ej. 300-3000 Hz)
- **MFCC:** Valores típicamente en el rango $[-200, 200]$
- **ZCR** (Zero Crossing Rate): Valores normalizados entre 0 y 1
- **HNR** (Harmonics-to-Noise Ratio): Valores en decibelios (dB)

Sin normalización, las características con mayor magnitud dominarían el cálculo de distancias, sesgando el modelo. Por ello, se aplicó `StandardScaler` para estandarizar las características a media cero y desviación típica uno:

Listing 9: Normalización de características para SVM

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Es crucial aplicar el escalado *después* de la división train/test para evitar *data leakage* (fuga de información del conjunto de prueba al entrenamiento).

Configuración experimental

El proceso experimental para SVM se estructuró en cuatro fases: (1) modelo baseline con kernel lineal, (2) modelo con kernel RBF, (3) validación cruzada, y (4) optimización de hiperparámetros mediante Grid Search.

Modelo baseline: SVM lineal Se estableció un primer modelo con kernel lineal como referencia simple:

Listing 10: Configuración SVM lineal baseline

```
svm_linear = SVC(
    kernel='linear',          # Frontera de decision lineal
    C=1.0,                   # Parametro de regularizacion
    probability=True,        # Habilita probabilidades para AUC-ROC
    random_state=12,
    class_weight='balanced'  # Compensacion de clases desbalanceadas
)
```

El parámetro `class_weight='balanced'` ajusta los pesos de las clases de forma inversamente proporcional a su frecuencia, compensando posibles desbalances en el conjunto de datos.

Modelo no lineal: SVM con kernel RBF Dada la naturaleza compleja de las señales de audio, se implementó un modelo con kernel RBF para capturar relaciones no lineales:

Listing 11: Configuración SVM con kernel RBF

```
svm_rbf = SVC(
    kernel='rbf',            # Kernel no lineal
    C=1.0,                   # Regularizacion
    gamma='scale',          # Coeficiente del kernel
    probability=True,
    random_state=12,
    class_weight='balanced'
)
```

El parámetro `gamma='scale'` establece automáticamente $\gamma = 1/(n_{\text{características}} \cdot \text{Var}(X))$, un valor inicial razonable.

Validación cruzada

Para garantizar la robustez de las evaluaciones y evitar dependencias de una única partición, se implementó validación cruzada estratificada con 5 folds. Se utilizó un pipeline que integra el escalado y el clasificador para asegurar que la normalización se realice correctamente en cada fold:

Listing 12: Pipeline con validación cruzada para SVM

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, cross_val_score

# Pipeline que integra escalado + SVM
svm_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', probability=True,
                random_state=42, class_weight='balanced'))
])
```

```

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=12)

cv_scores_accuracy = cross_val_score(svm_pipeline, X_train, y_train,
                                     cv=cv, scoring='accuracy')
cv_scores_f1 = cross_val_score(svm_pipeline, X_train, y_train,
                              cv=cv, scoring='f1')
cv_scores_auc = cross_val_score(svm_pipeline, X_train, y_train,
                              cv=cv, scoring='roc_auc')

```

El uso de pipeline es especialmente importante en SVM porque garantiza que la normalización se ajuste únicamente con los datos de entrenamiento de cada fold, evitando contaminación con datos de validación.

Optimización de hiperparámetros

SVM con kernel RBF tiene dos hiperparámetros críticos que determinan su rendimiento (ya explicados previamente):

- **C (parámetro de regularización).**
- **γ (gamma).**

La búsqueda sistemática se realizó mediante Grid Search con validación cruzada:

Listing 13: Grid Search para optimización de SVM

```

param_grid = {
    'svm__C': [0.1, 1, 10, 100],                # Regularizacion
    'svm__gamma': [0.001, 0.01, 0.1, 1, 'scale', 'auto'],
# Influencia del kernel
    'svm__kernel': ['rbf']                      # Nos centramos en RBF
}

grid_search = GridSearchCV(
    svm_pipeline,
    param_grid,
    cv=cv,
    scoring='f1',                               # Optimizamos F1-score
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_train, y_train)
svm_optimized = grid_search.best_estimator_

```

La elección de optimizar el F1-score responde a la necesidad de balancear precisión y sensibilidad en un problema donde ambos tipos de error (falsos positivos y falsos negativos) son relevantes.

Análisis de vectores de soporte

Una característica única de SVM es la identificación de los **vectores de soporte**: las muestras críticas que determinan la frontera de decisión. Su análisis proporciona información sobre la complejidad del problema:

Listing 14: Análisis de vectores de soporte

```
# Obtenemos el modelo del pipeline
svm_model = svm_optimized.named_steps['svm']
n_support_vectors = svm_model.n_support_
support_vectors_count = sum(n_support_vectors)

print(f"Total de vectores de soporte: {support_vectors_count}")
print(f"Porcentaje del dataset: {support_vectors_count/len(X_train)*100:.2f}%")
print(f"Vectores clase Real(0): {n_support_vectors[0]}")
print(f"Vectores clase IA(1): {n_support_vectors[1]}")
```

La interpretación de estos valores es reveladora:

- Un bajo porcentaje de vectores de soporte (< 30 %) indica que la frontera de decisión es relativamente simple y está bien definida por pocos ejemplos.
- Si una clase tiene significativamente más vectores de soporte, su región de decisión es más compleja o presenta mayor solapamiento.
- Los vectores de soporte son candidatos naturales para un análisis cualitativo: ¿qué tienen en común los audios más difíciles de clasificar?

Métricas de evaluación

Se emplearon las mismas métricas que en Random Forest y LightGBM:

- **Accuracy**: Proporción de predicciones correctas.
- **Precision**: Proporción de predicciones positivas correctas.
- **Recall**: Proporción de positivos reales correctamente identificados.
- **F1-Score**: Media armónica de precisión y recall.
- **AUC-ROC**: Área bajo la curva ROC.
- **MCC**: Coeficiente de correlación de Matthews.

La función de evaluación implementada es la misma que para los modelos anteriores.

3.4.6. AASIST

AASIST (*Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks*) es un modelo creado con la motivación de combinar pistas espectrales y temporales para detectar artefactos de spoofing y conseguir robustez frente a una gran

variedad de ataques. Como este modelo produce una predicción a partir del audio crudo por lo que el primer paso es definir las rutas a los audios.

Después, se lee el contenido del **archivo .conf** del modelo. Para ello, hemos creado una función que busca el elemento pasado por parámetro y devuelve el bloque superior que contiene ese elemento junto con los elementos al mismo nivel. Esto nos permite construir una variable, que vamos a llamar `args`, con los siguientes campos:

- **first_conv**: configuración del **kernel** de la primera capa de convolución, es decir, cuántas muestras de audio se utilizan para cada filtro. En este caso, 128.
- **filt**: es un array de elementos siendo el primero el número de **filtros** en la primera capa de convolución. Los siguientes elementos son pares para cada bloque residual con el número de filtros de entrada y de salida.
- **gat_dims**: tamaño del **embedding** que representa cada nodo. Tiene dos valores que hacen referencia al tamaño al construir los grafos de atención y al tamaño en las fases de “HS-GAL” + “pooling” respectivamente.
- **pool_ratios**: se usa en la técnica de “graph pooling” y significa el **ratio de reducción de nodos** en cada capa. Los dos primeros valores se usan para reducir los grafos espectral y temporal respectivamente antes de combinarlos en el grafo heterogéneo. El siguiente valor se usa para reducir el grafo heterogéneo después de cada capa HS-GAL. Aunque existe un cuarto valor, en el modelo final no se usa.
- **temperatures**: controla lo suave o agresiva que es la **atención** en cada fase. Cuanto más bajo es este parámetro, mayor peso se concentra en pocos nodos vecinos.

Con `args` ya creado, la usamos para instanciar el modelo. Posteriormente, se usa el checkpoint para cargar los pesos preentrenados.

El siguiente paso es definir las funciones de inferencia. Para esto, nos apoyamos primero en dos funciones auxiliares, una que convierte los audios a un **formato estándar**, un solo canal y 16 kHz, y otra que crea una **ventana** para recorrer audios largos. Esta última es importante ya que AASIST está diseñado para recibir segmentos de longitud de unos 4 segundos. Con esto preparado, las tres funciones de inferencia las creamos para ver si conseguimos mejores resultados alterando un un poco el proceso y se describen a continuación:

- **average_windows**: esta función devuelve la media de sumar las predicciones de cada ventana calculadas de forma independiente.
- **score_without_windows**: esta función devuelve solamente la puntuación del segmento inicial del audio en archivos largos o del audio en su totalidad en el caso contrario.
- **average_logits**: esta función fusiona todas las ventana sumando sus logits antes de calcular una única probabilidad.

Finalmente, se aplica el modelo y se vuelcan los resultados en un csv con el nombre del audio, la probabilidad de que sea “bonafide” y su duración. Para esto, se recorre cada audio del dataset, se le aplica una de las funciones de inferencia y se escribe la información en el csv.

Resultados y análisis Como hemos mencionado anteriormente, implementamos tres funciones de inferencia distintas para intentar obtener mejores resultados. Tras varias pruebas, alternando las tres funciones, obtuvimos resultados pésimos en los que el modelo tendía a clasificar cada muestra como “bona fide” y alguna como “spoof” pero sin mucho acierto lo que da una sensación de clasificación al azar. Ante estos resultados, decidimos realizar “**fine-tuning**” sobre el propio modelo preentrenado, nuestro propio preentrenamiento con parte del dataset que creamos. Para conseguir esto, primero separamos el dataset en train, eval y dev y generamos los archivos de “protocols” de donde el modelo va leyendo el nombre y el tipo de los audios de cada partición. Adicionalmente, tuvimos que hacer ciertos ajustes en el .conf y en especial, ajustar el número de épocas.

Para las pruebas, las hemos separado en tres tipos: audios en inglés por separado, español por separado y ambos idiomas juntos. Dentro de cada escenario, hay cuatro pruebas con distinto número de **épocas: 10, 20, 40 o 80**. También, cabe mencionar que siempre se ha mantenido una proporción en la separación en train, dev y eval de 72 %, 14 % y 14 %, respectivamente. Por último, antes de analizar los resultados, es importante explicar que en la separación manual del dataset hemos separado los distintos speakers que había en cada idioma. Esto se refiere a que en el dataset hay por idioma, 5 speakers distintos con 10 audios “bonafide” y 10 “spoof”, entonces, hemos intentado que en las tres particiones haya audios de las 5 personas y así intentar evitar sesgos.

A la hora de evaluar, hemos usado 3 medidas:

Precision: proporción de las muestras spoof detectadas sobre el total de muestras clasificadas como spoof

$\text{Precision} = \frac{TP}{TP+FP}$ **Recall:** proporción de muestras spoof detectadas correctamente sobre el total de muestras spoof

$$\text{Recall} = \frac{TP}{TP+FN}$$

F1: media armónica entre precision y recall

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

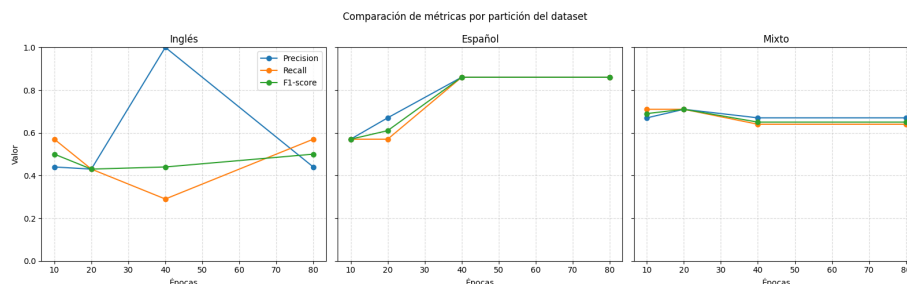


Figura 5: Resultados del preentrenamiento en AASIST.

Lo primero que podemos destacar sobre las gráficas, es que el modelo clasifica claramente mejor los audios en español ya que todas las métricas, a excepción de precisión en 40 épocas, tienen valores superiores. Esta diferencia explica el porqué la partición mixta lo hace peor que la que solo tiene muestras en inglés y permite deducir que ejecutar ambos idiomas juntos no le ayuda a clasificarlos mejor.

En lo referido a las épocas, lo más destacable es que superar las 40 épocas empeora la predicción por lo que podemos suponer que está ocurriendo un fenómeno de sobre-aprendizaje. También, podemos observar que en la partición en inglés, el aumento del valor de este parámetro no equivale en términos generales a una subida en la mejora de la predicción, fenómeno que no ocurre con la partición en español.

Explicabilidad Hemos usado las propias estructuras del modelo para empezar a explicar qué es lo que le empuja a decidir una clase a la otra. Esto nos sirve además para enseñar mejor cómo funciona AASIST por dentro. Lo primero que mostramos es el formato de la salida que es una tupla con dos valores, la probabilidad de que el audio sea “spoof” y de que sea “bona fide”, y la estructura de los grafos que nos permite ver que se crean 23 nodos espectrales y 29 temporales. También, simulamos como funciona un “pooling” mostrando los nodos de cada tipo con mayor puntuación.

Después, ejemplificamos cuales son los valores que se pasan a la capa de salida y cuánto afectan al cálculo de cada uno de los valores de la salida. Estos valores son los resultados de las operaciones “max” y “average” de cada subgrafo y el resultado del stack node. Con estas muestras podemos empezar a ver que ha sido mas significativo a la hora de responder si un audio es “spoof” o no.

En el apartado temporal, el siguiente paso es crear una función que va tapando segmentos de audio, calculamos la nueva probabilidad según el clasificador y vemos como cae la probabilidad base respecto a la nueva. Esta función de forma predeterminada hace estos cálculos para la clase predecida pero le puedes indicar tú por parámetro cualquiera de las dos. Las caídas en la probabilidad las graficamos y nos permite ver qué instantes del audio son más relevantes para el modelo. Aunque todavía no sabemos qué fenómenos son los determinantes si nos permite saber en qué momento ocurren.

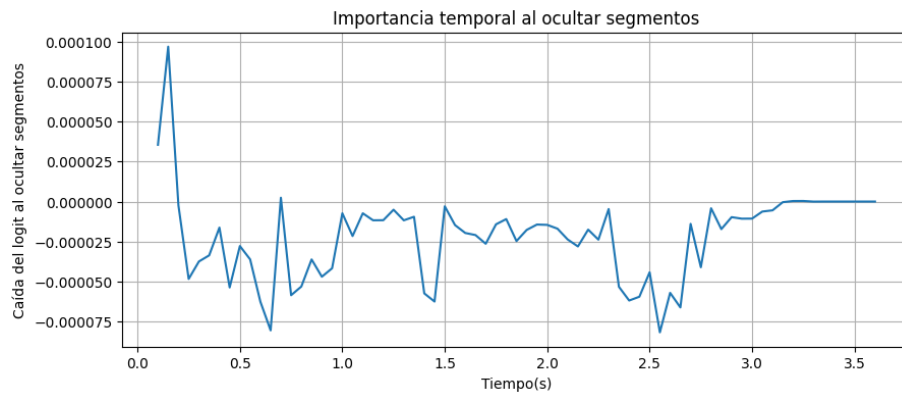


Figura 6: Resultados de la ocultación temporal en AASIST.

Como vemos en este ejemplo, en las primeras décimas de segundo ocurre un evento completamente determinante para el predictor. Destacamos los instantes con valor negativo que indican que si “quitamos” ese trozo la probabilidad crece.

En el ámbito espectral, primero calculamos la banda de frecuencias aproximada de cada uno de los 70 filtros con una transformada rápida de fourier y las graficamos

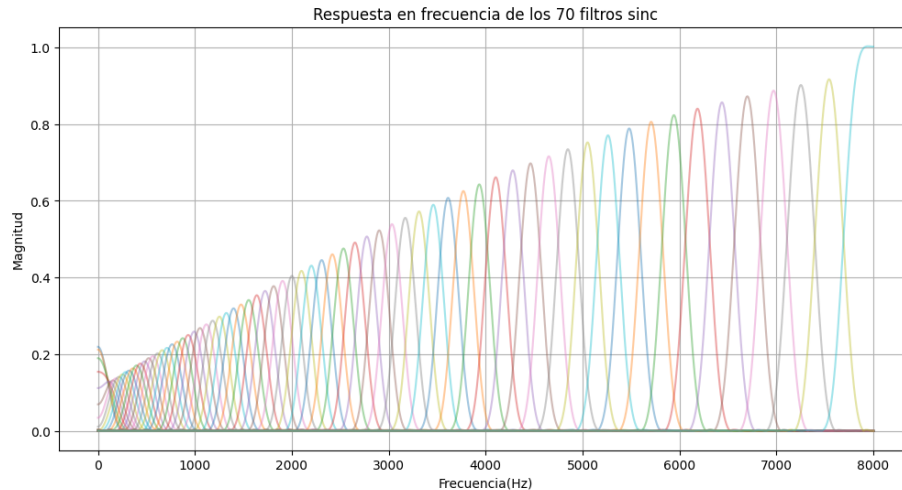
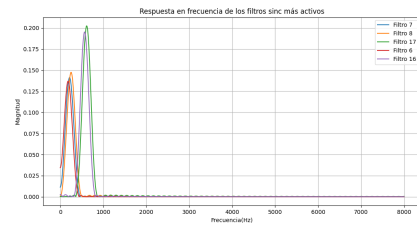


Figura 7: Gráfica bandas de frecuencia de los filtros.

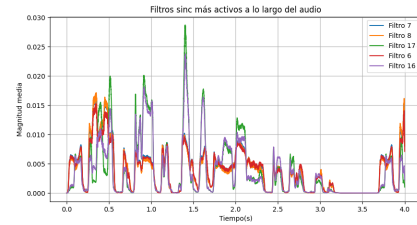
Con esta gráfica, podemos afirmar que se captan todas las frecuencias de 0 a 8000 Hz y que en las frecuencias bajas hay más solapamiento entre filtros. Esto es importante ya que cuando extraemos cuales son los filtros que más se activan durante el procesado del audio, no es raro que salgan algunos consecutivos ya que si dos filtros están observando

una frecuencia muy activa, ambos responderán parecido.

Posteriormente, graficamos los filtros que más activación han tenido en el primer procesado de dos formas: en la imagen de la izquierda viendo que frecuencias cubren y en la imagen de la derecha viendo su activación a lo largo del audio.



(a) Bandas de frecuencia de los filtros más activados.

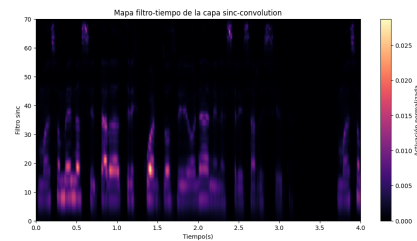


(b) Activación de los filtros más activados a lo largo del audio.

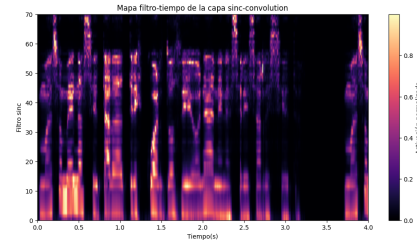
Como hemos mencionado anteriormente, los 5 filtros más excitados se pueden separar en dos grupos por su consecución. Además, en la gráfica de la derecha se ve claramente como tienen comportamientos muy parecidos. Comparando estas últimas imágenes, con la figura X podemos ver que donde en está última parece que había mucha relevancia, aquí no se ve reflejado. Esto se explica en que los filtros son solo la activación temprana del frontal sinc-conv y la información captada tiene que pasar por varios procesos. Sin embargo, la observación de los filtros empieza a ofrecernos información relevante para la clasificación.

Por último, en relación al ámbito espectral, simulamos como sería la salida de la capa sinc-convolution que podemos asemejar a un espectrograma. Para conseguir este mapa filtro-tiempo capturamos la salida de la capa durante la inferencia y la representamos en un “heatmap” en cuyo eje Y se colocan los filtros de menor a mayor banda de frecuencia.

Hemos creado dos mapas filtro-tiempo, el de la izquierda que sería el más fiel al modelo y el de la derecha que normaliza la activación de cada filtro por su máximo. Esta combinación nos permite determinar qué regiones de frecuencias se excitan más pero sin ocultar completamente a las menos determinantes.



(a) Mapa filtro-tiempo de la capa sinc-convolution.



(b) Mapa filtro-tiempo normalizado.

Con estos mapas, volvemos a observar que los filtros más activados son los de baja frecuencia, predominantemente en los primeros segundos del audio. Además, volvemos a ver una región “vacía” en torno a los 3,5 segundos que equivale a un silencio en el audio.

3.5. Modelos de explicación

3.5.1. Random Forest

Implementación de SHAP para la interpretabilidad del modelo Random Forest

Para complementar el análisis predictivo y dotar de interpretabilidad al modelo Random Forest, se implementaron explicaciones basadas en SHAP (SHapley Additive exPlanations). Esta sección describe la implementación práctica realizada sobre el modelo optimizado.

Funciones implementadas para explicabilidad

Se desarrollaron dos funciones principales para generar y visualizar explicaciones SHAP, tanto a nivel global como local.

Función de explicación SHAP La función `explica_shap` (Código 15) genera las explicaciones y visualizaciones:

Listing 15: Función para generar explicaciones SHAP

```
def explica_shap(modelo, X_train, X_test, n_show, local):
    # Creacion del explainer basado en el modelo
    explainer = shap.Explainer(modelo.predict, X_train)

    # Calculo de valores SHAP para el conjunto de prueba
    shap_values = explainer(X_test)

    # Visualizacion segun el tipo solicitado
    if local:
        # Explicaciones locales: diagramas de cascada (waterfall)
        # para las primeras n_show instancias
        for j in range(n_show):
            shap.plots.waterfall(shap_values[j])
    else:
        # Explicacion global: summary plot
        shap.summary_plot(shap_values, X_test)
        plt.show()

    # Retorno de predicciones para evaluacion posterior
    y_pred = modelo.predict(X_test)
```

```

y_pred_proba = modelo.predict_proba(X_test)[: , 1]
return y_pred , y_pred_proba

```

El funcionamiento de esta función es el siguiente:

1. Se crea un `shap.Explainer` utilizando el método de predicción del modelo y el conjunto de entrenamiento como referencia.
2. Se calculan los valores SHAP para todas las instancias del conjunto de prueba.
3. Dependiendo del parámetro `local`:
 - Si `local=True`: Se generan `n_show` diagramas de cascada (*waterfall plots*), cada uno mostrando la contribución de cada característica a la predicción de un audio específico.
 - Si `local=False`: Se genera un *summary plot* que agrega los valores SHAP de todas las instancias, mostrando la importancia global de cada característica y la dirección de su impacto.
4. La función retorna las predicciones (clases y probabilidades) para su posible evaluación posterior.

Función de validación mediante eliminación de características Para validar la importancia de las características identificadas por SHAP, se implementó la función `drop_features_explica_shap`, que permite realizar un experimento de ablación:

Listing 16: Función para evaluar impacto de eliminar características

```

def drop_features_explica_shap(modelo , model_name , X_train , y_train ,
                               X_test , y_test , feats , n_show ,
                               evalua_modelo=False , local=True):
    # Creacion de nuevos conjuntos sin las características especificadas
    X_train_new = X_train.drop(feats , axis=1, inplace=False)
    X_test_new = X_test.drop(feats , axis=1, inplace=False)

    # Reentrenamiento del modelo con los mismos hiperparametros
    params = modelo.get_params()

    modelo = RandomForestClassifier(**params)
    modelo.fit(X_train_new , y_train)

    # Generacion de explicaciones para el nuevo modelo
    y_pred , y_pred_proba = explica_shap(
        modelo=modelo ,
        X_train=X_train_new ,
        X_test=X_test_new ,
        n_show=n_show ,
        local=local
    )

```

```

)

# Evaluacion opcional del rendimiento
if evalua_modelo:
    evaluacion_modelo(
        model_name=model_name ,
        y_true=y_test ,
        y_pred=y_pred ,
        y_pred_proba=y_pred_proba
    )

return modelo_new , X_train_new , X_test_new

```

Esta función permite:

1. Eliminar un conjunto de características especificadas (**feats**) de los conjuntos de entrenamiento y prueba.
2. Reentrenar un nuevo modelo Random Forest con los mismos hiperparámetros que el modelo original.
3. Generar nuevas explicaciones SHAP para el modelo reducido.
4. Evaluar opcionalmente el rendimiento del nuevo modelo utilizando la función `evaluacion_modelo`, la cual calcula una serie de métricas como f1-score, accuracy o roc-auc, entre otras.

Interpretación de las visualizaciones generadas

Las visualizaciones obtenidas se interpretan de la siguiente manera:

- **Summary plot (global):** Muestra las características ordenadas por importancia. El color indica el valor de la característica (rojo = alto, azul = bajo) y la posición en el eje X indica el valor SHAP (impacto en la predicción). Características con puntos extendidos horizontalmente son más influyentes. Un ejemplo es:
- **Waterfall plot (local):** Para un audio específico, muestra:
 - $E[f(X)]$: Valor base (probabilidad media de la clase IA).
 - $f(x)$: Predicción final para este audio.
 - Barras rojas/azules: Características que empujan la predicción hacia IA (rojo) o hacia real (azul), con la magnitud de su contribución.

Un ejemplo de explicación local es:

Implementación de LIME para la interpretabilidad local del modelo

Como complemento a SHAP, se incorporó LIME (Local Interpretable Model-agnostic Explanations) para obtener explicaciones locales de las predicciones del modelo Ran-

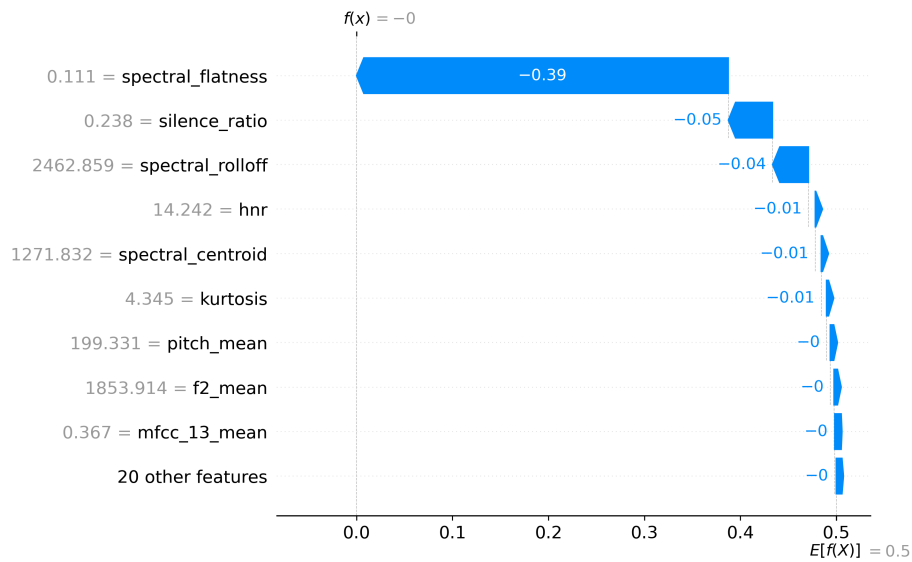


Figura 10: Explicación local con SHAP

dom Forest. LIME genera explicaciones interpretables aproximando localmente el modelo complejo mediante un modelo simple (lineal) en la vecindad de la instancia a explicar.

Funciones implementadas para explicaciones LIME

Para integrar LIME en el flujo de trabajo, se implementó un procedimiento que permite generar explicaciones visualmente claras y personalizar su presentación para facilitar su inclusión en informes y memorias.

Configuración del explainer LIME El primer paso consiste en crear un explainer de LIME adaptado a datos tabulares:

Listing 17: Configuración del explainer LIME

```
from lime.lime_tabular import LimeTabularExplainer
import re
from IPython.display import HTML, display
```

Creacion del explainer para datos tabulares

```
explainer = LimeTabularExplainer(
    X_train.values,
    feature_names=X.columns,
    class_names=['label'],
    mode='classification'
)
```

Datos de entrenamiento en formato numpy
Nombres de las características
Nombres de las clases (0: Real, 1: IA)
Modo de clasificacion

Los parámetros utilizados son:

- `X_train.values`: Matriz de características de entrenamiento en formato numpy array, necesaria para que LIME pueda muestrear la vecindad de las instancias.
- `feature_names`: Lista con los nombres de las características (formantes, MFCC, etc.) para etiquetar correctamente las explicaciones.
- `class_names`: Etiquetas descriptivas para las clases ('Real' y 'Ía') en lugar de valores numéricos.
- `mode='classification'`: Indica que se trata de un problema de clasificación.

Generación de explicaciones para una instancia Para obtener la explicación de un audio específico, se utiliza el método `explain_instance`:

Listing 18: Generación de explicación para una instancia

```
# Seleccionamos la primera instancia del conjunto de prueba como ejemplo
instancia_a_explicar = X_test.values[0]

# Generamos la explicacion
exp = explainer.explain_instance(
    instancia_a_explicar ,           # Instancia a explicar
    modelo_rf.predict_proba ,       # Funcion de prediccion (probabilidades
    num_features=10                 # Numero de caracteristicas a mostrar
)
```

El método `explain_instance` recibe:

- La instancia concreta a explicar (en formato numpy array).
- La función de predicción del modelo (en este caso `predict_proba`), que LIME utiliza para muestrear la vecindad y entrenar el modelo local.
- `num_features`: Número de características más relevantes a incluir en la explicación (por defecto 10).

Personalización visual de las explicaciones LIME

LIME genera por defecto visualizaciones con estilos que pueden no integrarse adecuadamente en el formato de la memoria. Para solucionarlo, se implementó un proceso de limpieza y personalización del HTML generado:

Listing 19: Limpieza y personalización del HTML de LIME

```
# Obtenemos el HTML generado por LIME
exp_html = exp.as_html()

# Removemos las etiquetas <style> internas que pueden forzar estilos no deseados
exp_html_clean = re.sub(
```

```

        r'<style .*?>.*?</style>',
        '',
        exp_html,
        flags=re.DOTALL
    )

# Envolvemos el HTML limpio en un contenedor con estilos personalizados
html_con_style = f"""
<style>
.lime-container * {{
    color: black !important;          # Texto en negro
    background-color: white !important; # Fondo blanco
    border-color: black !important;    # Bordes en negro
}}
</style>
<div class="lime-container">
    {exp_html_clean}
</div>
"""

# Mostramos el resultado
display(HTML(html_con_style))

```

Este proceso realiza las siguientes transformaciones:

1. **Extracción del HTML:** `exp.as_html()` genera una representación HTML completa de la explicación LIME, incluyendo estilos CSS embebidos.
2. **Limpieza de estilos:** Mediante una expresión regular (`re.sub`) se eliminan todas las etiquetas `<style>` internas que LIME incluye por defecto. Esto evita conflictos con los estilos de la memoria y permite un control total sobre la apariencia.
3. **Envoltura con estilos personalizados:** Se crea un contenedor `<div>` con clase `lime-container` y se definen estilos CSS que fuerzan:
 - Texto en color negro (`color: black !important`)
 - Fondo blanco (`background-color: white !important`)
 - Bordes en negro (`border-color: black !important`)

El uso de `!important` garantiza que estos estilos prevalezcan sobre cualquier otro.

4. **Visualización:** Se utiliza `display(HTML(...))` de IPython para renderizar el HTML personalizado en el notebook.

Interpretación de las explicaciones LIME

La visualización generada por LIME muestra:

- **Predicción del modelo:** La probabilidad asignada a cada clase (Real/IA) para la instancia explicada.
- **Características más influyentes:** Lista de las `num_features` características con mayor impacto en la decisión, ordenadas por importancia.
- **Contribución de cada característica:** Barras horizontales que indican si la característica apoya la clase IA (color naranja/rojo) o la clase Real (color azul), junto con la magnitud de su contribución.
- **Valor de la característica:** Se muestra el valor concreto que tenía esa característica en el audio explicado.

Consideraciones sobre la implementación

Durante la implementación se tuvieron en cuenta los siguientes aspectos:

- **Formato de datos:** LIME requiere los datos en formato numpy array (`.values`) en lugar de DataFrame de pandas, por lo que se realiza la conversión explícita.
- **Función de predicción:** Se utiliza `predict_proba` en lugar de `predict` para obtener las probabilidades, lo que permite a LIME capturar la confianza del modelo y no solo la clase final.
- **Número de características:** Se limitó a 8-10 características por explicación para mantener la legibilidad, aunque el parámetro es configurable.
- **Personalización visual:** La limpieza de estilos CSS garantiza que las explicaciones LIME mantengan una apariencia profesional y consistente con el resto de la memoria, evitando problemas de contraste o colores no deseados.

Implementación de DiCE para la generación de explicaciones contrafactuales

Para complementar las explicaciones basadas en atribución (SHAP y LIME), se incorporó DiCE (Diverse Counterfactual Explanations) para generar explicaciones contrafactuales. DiCE responde a la pregunta: ¿qué cambios mínimos en las características de un audio clasificado como “a” lo harían ser clasificado como “b”, y viceversa?

Preparación de datos para DiCE

DiCE requiere los datos en un formato específico, combinando características y etiqueta en un mismo DataFrame:

Listing 20: Preparación de datos para DiCE

```
# Eliminacion de columnas no relevantes (ej. filename)
X_train_clean = X_train.drop(columns=['filename'], errors='ignore')
X_test_clean = X_test.drop(columns=['filename'], errors='ignore')
```

```
# Combinacion de características con la variable objetivo
data_dice = X_train_clean.copy()
data_dice['label'] = y_train # Variable objetivo (0: Real, 1: IA)
```

Los pasos realizados son:

- **Limpieza de características no predictivas:** Se elimina la columna `filename` (si existe) ya que contiene identificadores de archivo sin valor predictivo y no tendría sentido generar contrafactuales modificándola.
- **Integración de la variable objetivo:** DiCE necesita conocer la relación entre características y etiqueta, por lo que se añade la columna `label` al DataFrame de entrenamiento.

Configuración de DiCE

DiCE utiliza dos componentes principales: un `DataInterface` para manejar los datos y un `Model` para encapsular el clasificador:

Listing 21: Configuración de DiCE

```
import dice_ml

# Definicion del DataInterface
dice_data = dice_ml.Data(
    dataframe=data_dice, # DataFrame con features +
    continuous_features=X_train_clean.columns.tolist(), # Todas las features son
    outcome_name='label' # Nombre de la variable o
)

# Encapsulacion del modelo
dice_model = dice_ml.Model(
    model=modelo_rf, # Modelo Random Forest entrenado
    backend='sklearn' # Backend para modelos scikit-learn
)

# Creacion del explicador DiCE
exp = dice_ml.Dice(
    dice_data, # DataInterface
    dice_model, # Model encapsulado
    method="random" # Metodo de generacion ("random" o "genetic")
)
```

Los parámetros utilizados son:

- **dataframe:** DataFrame completo que contiene tanto las características como la variable objetivo para el entrenamiento.

- **continuous_features**: Lista de nombres de todas las características continuas. En este caso, todas las características de audio (formantes, MFCC, etc.) son continuas.
- **outcome_name**: Nombre de la columna que contiene la variable objetivo ('label').
- **backend='sklearn'**: Indica que el modelo es de scikit-learn, permitiendo a DiCE acceder a los métodos necesarios.
- **method='random'**: Método de generación de contrafactuales. `random`.^{es} más rápido; también existe "genetic" para búsquedas más exhaustivas.

Generación de contrafactuales para una instancia

Una vez configurado DiCE, se genera un conjunto de contrafactuales para una instancia específica:

Listing 22: Generación de contrafactuales

```
# Selección de la instancia a explicar (primera del conjunto de prueba)
query = X_test_clean.iloc[[0]] # Usamos iloc para mantener estructura DataFrame

# Generación de contrafactuales
e1 = exp.generate_counterfactuals(
    query_instances=query,      # Instancia(s) a explicar
    total_CFs=3,               # Número de contrafactuales a generar
    desired_class="opposite"    # Clase deseada (opuesta a la predicción original)
)

# Visualización de resultados
e1.visualize_as_dataframe(show_only_changes=True)
```

Los parámetros de generación son:

- **query_instances**: DataFrame con una o más instancias para las que generar contrafactuales. Se utiliza `iloc[[0]]` para mantener la estructura de DataFrame (a diferencia de `iloc[0]` que devolvería una Serie).
- **total_CFs**: Número de contrafactuales a generar (en este caso 3), permitiendo explorar diferentes alternativas para cambiar la clasificación.
- **desired_class**: Clase objetivo para los contrafactuales. `"opposite"` indica la clase contraria a la predicción original. También se puede especificar directamente 0 (Real) o 1 (IA).
- **show_only_changes=True**: En la visualización, muestra únicamente las características que cambian respecto a la instancia original, facilitando la identificación rápida de las modificaciones necesarias.

Interpretación de los contrafactuales generados

La visualización generada por `visualize_as_dataframe(show_only_changes=True)` muestra:

- **Instancia original:** Valores de las características para el audio consultado y su clasificación original.
- **Contrafactuales:** Para cada uno de los `total_CFs` generados:
 - Valores de las características que cambian respecto al original (si `show_only_changes=True`).
 - La nueva clasificación resultante (clase deseada).
 - La magnitud del cambio necesario en cada característica.
- **Diversidad:** DiCE genera contrafactuales diversos, mostrando diferentes formas de lograr el mismo cambio de clasificación (por ejemplo, modificando diferentes combinaciones de formantes o MFCC).

La salida para la primera instancia es:

Interpretación en el contexto de detección de deepfakes

Las explicaciones contrafactuales permiten responder preguntas como:

- **Para un audio clasificado como IA:** "¿Qué características acústicas deberían cambiar para que el modelo lo considere voz real, y en qué magnitud?"
- **Para un audio clasificado como real:** "¿Qué modificaciones lo harían ser clasificado como generado por IA?"
- **Vías alternativas:** DiCE muestra múltiples formas de lograr el cambio, revelando si hay “camino” principales (modificar ciertos formantes) o alternativos (modificar MFCC específicos).

Consideraciones sobre la implementación

Durante la implementación se tuvieron en cuenta los siguientes aspectos:

- **Eliminación de columnas no relevantes:** Características como `'filename'` se eliminan porque no tienen sentido en un contrafactual (no se puede modificar el nombre del archivo para cambiar la clasificación).
- **Selección de instancia:** Se utiliza `iloc[[index]]` en lugar de `iloc[index]` para mantener la estructura de `DataFrame` requerida por DiCE.
- **Método de generación:** Se eligió `method=random` por su eficiencia computacional, aunque para análisis más exhaustivos se puede utilizar `method=genetic`.
- **Visualización enfocada:** El parámetro `show_only_changes=True` facilita la interpretación al mostrar únicamente las características que realmente cambian, evitando abrumar con información redundante.

- **Clase deseada:** El valor `opposite` es útil cuando se quiere explorar el cambio de clase, pero también se puede especificar explícitamente `0` o `1` si se busca una dirección concreta.